

CHAPTER-1

INTRODUCTION

1.1 INTRODUCTION

Weather conditions play a crucial role in shaping various sectors such as agriculture, transportation, urban planning, and disaster management. Accurate and timely weather monitoring is essential for mitigating risks and optimizing operations in these fields. Traditionally, weather monitoring relied on manual observations and stationary weather stations, which often fell short in providing real-time data and comprehensive coverage. However, the advent of **Internet of Things (IoT)** technologies has revolutionized weather monitoring by enabling the deployment of interconnected sensors capable of collecting real-time environmental data from diverse geographical locations.

Moreover, the integration of **machine learning (ML)** algorithms into weather forecasting systems has significantly enhanced the accuracy and predictability of weather models. By leveraging historical weather data and advanced computational techniques, ML models can analyze complex patterns, detect anomalies, and provide more precise forecasts. This synergy of IoT and ML technologies addresses the limitations of traditional weather monitoring methods, paving the way for more resilient and efficient weather management systems.

1.2 CHALLENGES IN TRADITIONAL WEATHER MONITORING

Traditional weather monitoring systems face several limitations that hinder their effectiveness:

1. **Manual Data Collection:** Reliance on manual methods is prone to errors, delays, and inconsistencies.
2. **Limited Coverage:** Fixed weather stations provide localized data, often missing broader regional patterns.
3. **Lack of Real-Time Data:** Delays in data acquisition and processing limit the ability to respond promptly to changing conditions.
4. **Data Integration Issues:** Disparate systems lack seamless connectivity, making it challenging to consolidate and analyze data comprehensively.

These challenges underscore the need for advanced technologies like IoT and ML to enhance the scope, accuracy, and responsiveness of weather monitoring systems.

1.3 THE ROLE OF IOT IN WEATHER MONITORING

IoT technology transforms weather monitoring by enabling a network of interconnected sensors that continuously collect data on key environmental parameters such as temperature, humidity, wind speed, and precipitation. These sensors are strategically deployed across diverse locations to ensure comprehensive coverage.

Key benefits of IoT-based weather monitoring systems include:

1. **Scalability:** Easily expandable to cover larger areas.
2. **Flexibility:** Adaptable to various environmental conditions and applications.
3. **Real-Time Data Acquisition:** Provides up-to-the-minute insights for timely decision-making.
4. **Enhanced Resilience:** Facilitates proactive responses to weather-related risks.

1.4 THE ROLE OF MACHINE LEARNING IN WEATHER FORECASTING

While IoT enables the collection of vast amounts of environmental data, **machine learning (ML)** algorithms play a pivotal role in analyzing this data to generate accurate weather forecasts. ML models can identify patterns, predict trends, and improve forecast accuracy over time by learning from historical data.

Common ML algorithms used in weather forecasting include:

1. **Linear Regression & Polynomial Regression:** For predicting continuous variables like temperature and precipitation.
2. **Random Forest Regression:** For handling complex, non-linear relationships in weather data.
3. **Artificial Neural Networks (ANNs) & Recurrent Neural Networks (RNNs):** Ideal for modeling temporal patterns and time-series data, such as predicting weather conditions over several days.

These algorithms are trained using historical weather data, with a typical workflow involving:

- **Training Phase:** Using 80% of historical data to train the model.
- **Testing Phase:** Using the remaining 20% to validate the model's accuracy.

1.5 OBJECTIVES OF THE REPORT

The primary objective of this project is to develop an integrated weather monitoring and forecasting system that leverages IoT and machine learning technologies to provide real-time, accurate weather data. The project aims to:

1. **Deploy IoT-enabled Sensors:** Design and install a network of sensors to capture real-time environmental data from diverse locations.
2. **Implement Data Processing Algorithms:** Develop ML models to analyze the collected data for weather forecasting and anomaly detection.
3. **Centralized Data Integration:** Create a unified platform for easy access, visualization, and analysis of weather data by stakeholders such as researchers, policymakers, and the public.
4. **Enhance System Scalability and Security:** Ensure the system is robust, reliable, and secure for continuous operation and data integrity.
5. **Develop User-Friendly Interfaces:** Design interactive dashboards and mobile applications to visualize weather data intuitively.
6. **Collaborate with Stakeholders:** Work with meteorological agencies, academic institutions, and industry partners to validate the system's accuracy and effectiveness.

1.6 METHODOLOGY

This project aims to predict the temperature of **Bhopal City** using historical weather data and advanced machine learning algorithms. The dataset utilized for this analysis has been sourced from **Kaggle**, specifically the dataset titled "**Historical Weather Data for Indian Cities.**" This dataset includes hourly weather data from **January 1, 2013, to January 1, 2024**, covering over a decade of weather patterns.

The data for Bhopal was extracted using the **WorldWeatherOnline.com API** and processed with the help of the **wwo_hist package**. While the dataset provides extensive historical data, it's important to note that the accuracy of the data cannot be fully guaranteed due to the limitations of the data source.

1.6.1 DATA DESCRIPTION

- **Source:** Kaggle – "Historical Weather Data for Indian Cities"
- **Target City:** Bhopal, India
- **Date Range:** January 1, 2013 – January 1, 2024
- **Data Frequency:** Hourly weather data
- **Data Parameters:** Temperature, humidity, precipitation, wind speed, and other meteorological factors

This dataset serves dual purposes:

- **Weather Prediction:** Forecasting future weather conditions, including daily, weekly, and seasonal trends.
- **Climate Analysis:** Visualizing changes in weather patterns over time, aiding in the study of **global warming** and its effects on temperature, precipitation, and humidity.

1.6.2 TARGET and OBJECTIVES

The primary focus of this project is to **predict the temperature of Bhopal City** for future dates, specifically aiming to enhance the accuracy of short-term weather forecasts. The dataset's rich historical records enable the application of various machine learning algorithms to improve predictive performance.

Key Objectives:

- Predict daily and weekly temperature trends for Bhopal City.
- Analyze the impact of global warming through long-term data visualization.
- Compare the performance of different regression algorithms in temperature forecasting.

1.6.3 MACHINE LEARNING ALGORITHMS for TEMPERATURE PREDICTION

To achieve accurate temperature predictions, the project employs a combination of regression algorithms, each offering unique strengths in modeling weather data patterns.

1. Multiple Linear Regression (MLR):

- A statistical method used to model the relationship between multiple independent variables (e.g., humidity, pressure, wind speed) and the dependent variable (temperature).
- Provides a baseline for comparing the performance of more complex models.

2. Decision Tree Regression (DTR):

- A non-linear model that splits the data into branches based on decision rules, making it suitable for capturing complex patterns in weather data.
- Handles both numerical and categorical data effectively.

3. Random Forest Regression (RFR):

- An ensemble learning method that builds multiple decision trees and averages their predictions to improve accuracy and reduce overfitting.
- Particularly effective for large datasets with multiple features, as it manages high-dimensional data efficiently.

1.6.4 DATA PROCESSING WORKFLOW

The methodology follows a structured approach to ensure robust and reliable predictions:

1. Data Collection:

- Historical weather data for Bhopal City was extracted using the **WorldWeatherOnline API**.
- The data includes key weather parameters, focusing on temperature trends over the past decade.

2. Data Preprocessing:

- Handling missing values, outlier detection, and data normalization.
- Feature engineering to create new variables that enhance model performance.

3. Model Training and Evaluation:

- **Training Phase:** 80% of the dataset (from 2013 to 2021) is used to train the models.
- **Testing Phase:** The remaining 20% (2022 to 2024) is reserved for testing and validating model accuracy.
- **Performance Metrics:** Evaluation of models using metrics such as **Mean Absolute Error (MAE)**, **Root Mean Squared Error (RMSE)**, and **R² Score** to compare predictive accuracy.

4. Prediction and Visualization:

- Applying trained models to predict the temperature for **January 1, 2025**, and future dates.
- Visualization of predicted temperature trends using interactive graphs and dashboards.

1.6.5 POTENTIAL APPLICATIONS

The outcomes of this project can be applied in several domains:

1. **Weather Forecasting:** Providing accurate daily and weekly temperature predictions.
2. **Climate Change Studies:** Analyzing historical weather patterns to understand the effects of global warming.
3. **Disaster Management:** Predicting extreme weather events to support early warning systems.
4. **Agriculture and Energy Sector:** Assisting in resource planning and risk management.

CHAPTER-2

SOFTWARE AND HARDWARE REQUIREMENTS

2.1 INTRODUCTION

The software and hardware components of a computer system are required to use websites efficiently. The website manufacturer will list the system requirements on the package. If your computer system does not meet the system requirements then the website may not work correctly. System requirements for operating systems will be hardware components, while other application software will list both hardware and operating system requirements and Browser. System requirements are most commonly seen listed as minimum and recommended requirements. The minimum system requirements need to be met for the web application to run at all on your system, & the recommended system requirements, if met, will offer better software usability.

2.2 SOFTWARE REQUIREMENTS

Required software to work on this project: Developer Requirements:

- Frontend Technologies:
 - HTML (5)
 - CSS (3)
 - JavaScript
 - API
- Machine Learning:
 - Python
- Development Tools:
 - Visual Studio Code
 - ThingSpeak Platform
 - Arduino IDE
 - Wokwi
 - GitHub
- User Requirements:
 - Compatible Browser
 - Internet Connection

2.3 HARDWARE REQUIREMENTS

The following are the hardware requirements of the project:

- Nodemcu board
- Rain sensor
- BMP180 sensor
- DHT11 sensor

CHAPTER-3

PROBLEM DESCRIPTION

3.1 OVERVIEW

The primary objective of this project is to develop an integrated weather monitoring system that leverages IoT technologies to provide real-time and accurate weather data. The project aims to address the limitations of traditional weather monitoring methods by deploying a network of IoT-enabled sensors capable of collecting data on various environmental parameters such as temperature, humidity, wind speed, and precipitation.

Weather conditions significantly impact agriculture, transportation, urban planning, and disaster management. However, traditional weather monitoring systems, which rely heavily on manual observations and stationary weather stations, face critical limitations. These systems struggle to provide real-time data, offer comprehensive geographic coverage, and integrate seamlessly with modern technological solutions.

Key challenges in traditional weather monitoring include:

1. **Dependence on Manual Data Collection:** Manual processes are prone to human error, delays, and inefficiencies, reducing the accuracy and timeliness of data.
2. **Limited Geographic Coverage:** The localized nature of stationary weather stations restricts their ability to capture broader weather patterns, leading to incomplete insights.
3. **Lack of Connectivity and Interoperability:** Disparate systems often lack seamless connectivity, making integration and comprehensive analysis of data challenging.
4. **Inadequate Real-Time Monitoring:** Traditional systems are unable to frequently and promptly acquire data, which limits their effectiveness in real-time monitoring.

These limitations hinder the ability to manage weather-related challenges effectively, posing risks to critical sectors and overall resilience.

3.2 ADVANTAGES OF IOT BASED WEATHER MONITORING

1. **Real-Time Data Collection:** IoT devices provide continuous, automated data on weather parameters like temperature, humidity, and precipitation, enabling immediate analysis and decision-making.
2. **Enhanced Geographic Coverage:** IoT sensors can be deployed in remote and inaccessible locations, providing comprehensive data from regions where traditional weather stations are scarce or absent.
3. **Seamless Integration and Connectivity:** IoT networks enable efficient data sharing across platforms, ensuring interoperability and fostering collaboration among different agencies and stakeholders.
4. **Improved Accuracy and Efficiency:** Automated sensors reduce human error, while advanced analytics and AI-driven algorithms process large datasets to generate more precise and actionable insights.

CHAPTER-4

LITERATURE SURVEY

The convergence of **Internet of Things (IoT)** and **Machine Learning (ML)** technologies has significantly advanced the field of weather monitoring and forecasting. Traditional meteorological systems, often limited by static, costly infrastructure and low spatial coverage, are being rapidly transformed by real-time, scalable, and intelligent systems. This literature survey synthesizes developments in IoT-based weather data acquisition and ML-driven forecasting, drawing from recent literature and the report “*A Machine Learning Model for Weather Forecasting*”.

4.1 IOT-BASED TECHNOLOGIES AND THEIR ROLE IN WEATHER MONITORING

IoT refers to a network of interconnected devices capable of autonomous communication. In the context of weather monitoring:

- **IoT Devices** (e.g., DHT11, BMP180, rain sensors) gather real-time environmental data such as temperature, humidity, pressure, and rainfall.
- **Wireless Communication** protocols (e.g., Wi-Fi, Zigbee) transmit data to the cloud or edge devices
- **Cloud Computing** and **Edge Processing** allow remote monitoring and instant responses, especially in critical environments like agriculture and flood-prone areas.

These capabilities align with the *ML report*’s emphasis on **the need for real-time, high-resolution weather data**—which IoT devices are designed to provide.

4.2 INTEGRATION of MACHINE LEARNING In FORECASTING

The uploaded ML report introduces an **ML-based weather prediction system** that utilizes historical weather datasets and supervised learning algorithms (e.g., linear regression, decision trees, and ensemble methods). Key insights include:

- **Data Collection** from historical sources is essential for training accurate models.
- **Feature Selection and Preprocessing** are used to refine input variables (e.g., humidity, temperature, pressure) for optimal model training.
- **Model Accuracy** was evaluated using metrics like RMSE and MAE, demonstrating that ML models can achieve competitive forecasting accuracy when supplied with reliable data.

4.3 CHALLENGES and CONSIDERATIONS

Both IoT and ML face overlapping technical challenges:

- **Data Accuracy:** IoT sensors must be calibrated regularly; ML models depend on clean, consistent input data.
- **Infrastructure and Scalability:** Large-scale IoT deployment requires efficient networking; ML training needs computational resources.
- **Security and Privacy:** Both systems process sensitive environmental or location data.
- **Model Reliability:** As noted in the ML report, even well-trained models must be updated with fresh data—highlighting the importance of ongoing IoT data feeds.

4.4 FUTURE DIRECTIONS

The fusion of **IoT + ML** is paving the way for **autonomous weather intelligence systems**:

- **Edge AI:** Processing sensor data directly on edge devices using lightweight ML models.
- **Adaptive Forecasting Models:** Continuously learning from IoT data streams.
- **Smart City Integration:** Using urban weather data for traffic control, pollution monitoring, and infrastructure management.

The literature makes it clear that **IoT provides the data backbone**, while **ML provides the brain**—together forming a powerful ecosystem for modern weather intelligence.

CHAPTER-5

SOFTWARE REQUIREMENTS SPECIFICATION

5.1 FUNCTIONAL REQUIREMENTS

- Data Collection
- Data Processing
- Integration
- Alerting and Notification Systems

Each Functional requirement is described below:

1.Data Collection:

The system ensures the continuous collection of weather data through IoT-enabled sensors distributed across diverse locations. These sensors are capable of monitoring a wide array of parameters including temperature, humidity, wind speed, and air quality. To bolster the reliability and accuracy of collected data, the system implements advanced mechanisms such as automated quality control and sensor calibration.

2.Data Processing:

Real-time processing and analysis of collected data are pivotal components of the system's functionality. It employs sophisticated algorithms for detecting patterns, trends, and anomalies within the data. Furthermore, the system leverages historical data and external sources such as satellite imagery and weather models to enhance the accuracy and reliability of its forecasts.

3.Integration:

Integration plays a crucial role in the system's architecture, facilitating the seamless aggregation of data from multiple sensors and sources into a centralized platform. It boasts compatibility with existing weather monitoring systems and databases.

4.Alerting and Notification Systems:

Develop a smart, real-time alerting system that evaluates key environmental conditions such as air temperature, humidity, and external temperature. Automatically notify users of any significant weather anomalies, including extreme moisture or unsafe climate indicators.

Allow users to define personalized thresholds for triggering alerts based on specific conditions (e.g., temperature drops, excessive humidity). Notifications should include both impacts and actionable suggestions, such as improving drainage or focusing on climate resilience strategies. Deliver alerts through multiple channels, including email, SMS, and push notifications, to ensure users are informed promptly and can respond accordingly.

5.2 NON-FUNCTIONAL REQUIREMENTS

- Performance
- Security
- Compatibility
- Usability
- Scalability

Each Non-Functional requirement is described below:

1.Performance:

The system is designed to scale effortlessly to accommodate large volumes of data and an increasing number of sensors. It boasts low latency and high throughput for real-time data processing and analysis.

2.Security:

Robust security measures are implemented to safeguard sensitive weather data from unauthorized access and tampering. Data transmission channels are encrypted to ensure confidentiality and integrity.

3. Compatibility:

The system prioritizes compatibility with various communication protocols and standards, fostering interoperability with existing systems.

4. Usability:

Conduct usability testing to evaluate the system's ease of use and user satisfaction. Incorporate user feedback to iteratively improve the system's usability and user experience. Ensure accessibility features are implemented to accommodate users with diverse needs and disabilities.

5. Scalability:

Conduct scalability testing to assess the system's ability to handle increasing data volumes and user loads. Implement horizontal scaling techniques such as load balancing and clustering to distribute workload across multiple servers. Monitor system performance and scalability metrics to proactively identify and address scalability bottlenecks.

CHAPTER-6

SOFTWARE AND HARDWARE DESIGN

6.1 ARCHITECTURE

Architecture refers to the high-level design and structure of the system, outlining its components and their interactions. It serves as a blueprint that guides the development process, ensuring all parts of the system work together effectively. The architecture includes defining hardware and software components, communication protocols, data flow, and integration points, providing a framework for scalability, reliability, and performance. In this project, the architecture includes sensor integration, data transmission via microcontroller, cloud storage, and user access via a personal website.

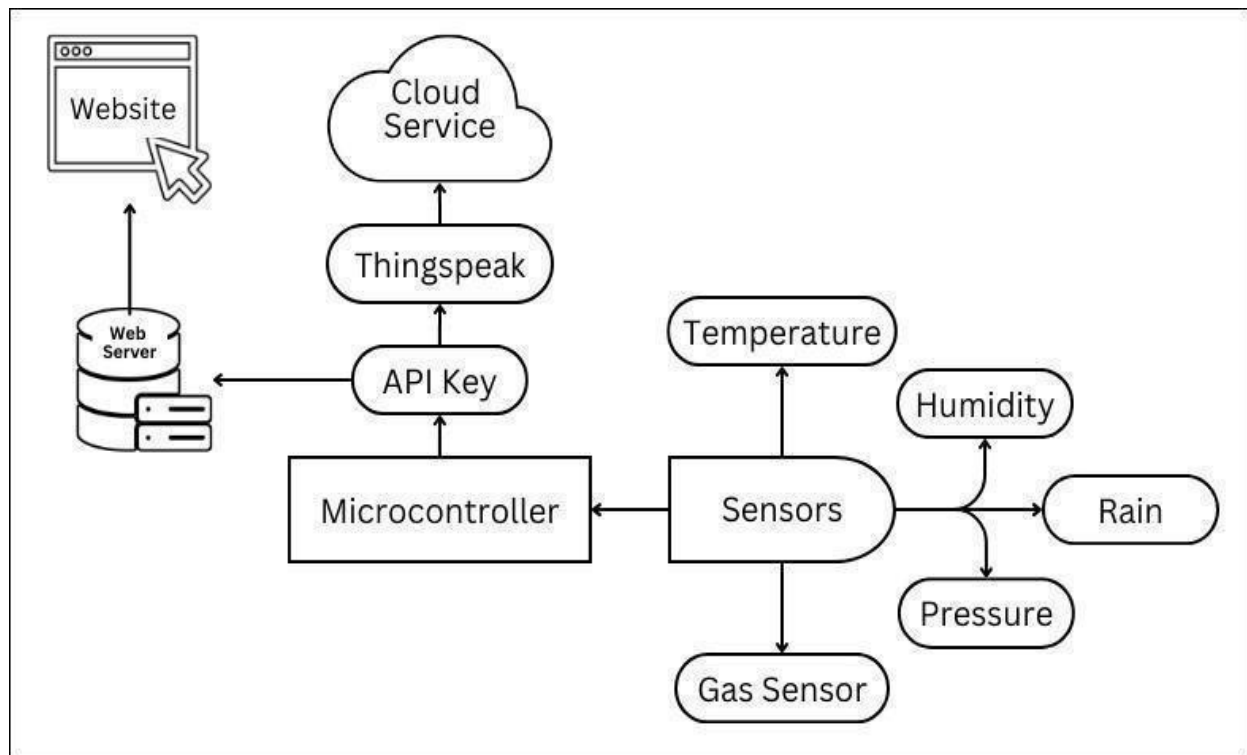


Fig 6.1: Architecture of Project Implementation

6.2 DATA FLOW DIAGRAM (DFD)

Data Flow Diagram (DFD) is a visual representation of a system's data flow and processes. It uses symbols like circles for processes, rectangles for data stores, and arrows for data flow. Processes transform input data into output data, while external entities interact with the system. Data stores represent where information is stored. DFDs aid in understanding system functionality, data movement, and high-level system operations. They are crucial in system analysis and design for modeling and documenting data flow.

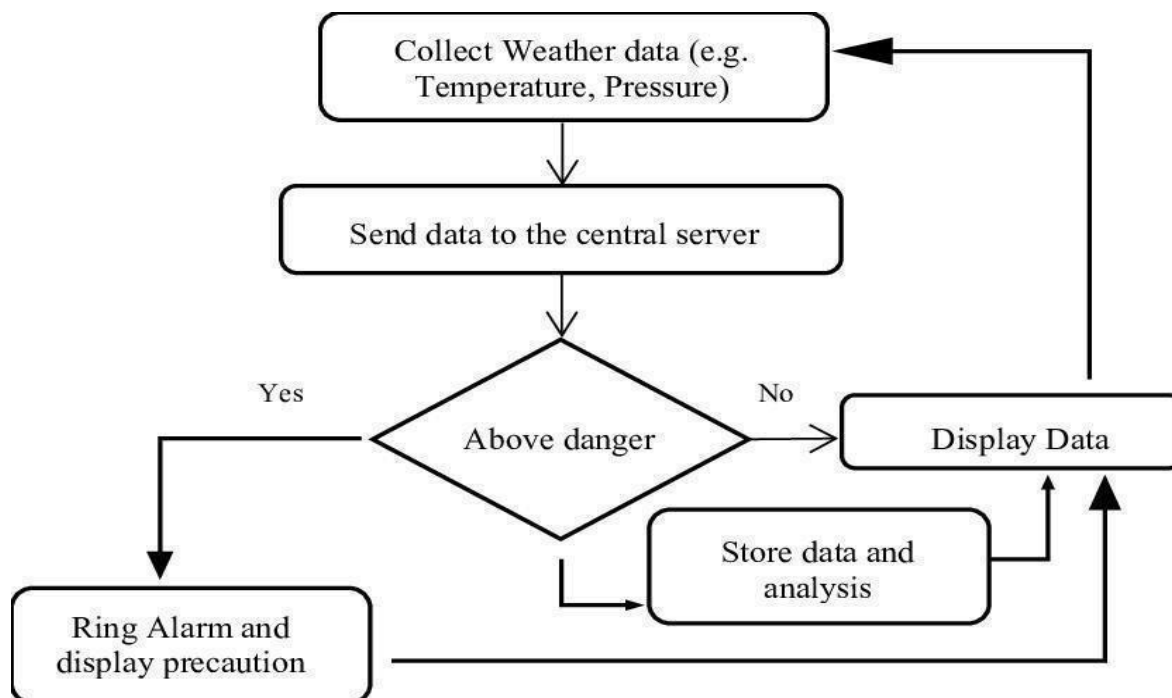


Fig. 6.2 Data Flow Diagram

CHAPTER-7

IOT MODULE

7.1 PRE-PROCESSING STEPS

IoT technology revolutionizes weather monitoring by establishing a network of interconnected sensors and devices capable of gathering diverse environmental data in real-time. These sensors, spanning temperature, humidity, pressure, wind speed, and precipitation, offer comprehensive insights into weather conditions. IoT-based weather monitoring systems offer scalability, flexibility, and accessibility, addressing the limitations of traditional methods and paving the way for enhanced resilience and efficiency in managing weather-related challenges.

Leveraging IoT connectivity, the collected data transmits wirelessly to a centralized platform for analysis and interpretation. This seamless flow of information empowers stakeholders with up-to-date weather insights from varied geographical locations, enhancing decision-making processes. By harnessing the power of IoT, weather monitoring systems unlock new possibilities for understanding and managing weather phenomena, ultimately improving resilience and sustainability across sectors.

7.2 DATA VISUALIZATION

In the rapidly evolving field of data analysis and visualization, researchers and practitioners have devoted significant attention to the development of advanced methodologies for processing and interpreting weather data collected by Internet of Things (IoT) sensors. These sensors, which are often deployed across a wide range of geographic locations, generate vast amounts of real-time environmental data, including temperature, humidity, wind speed, atmospheric pressure, and precipitation levels. To effectively analyse this complex and voluminous data, researchers have turned to cutting-edge machine learning algorithms and statistical models capable of identifying hidden patterns, making accurate forecasts, and detecting anomalies in weather behaviour.

Machine learning techniques such as deep learning, neural networks, decision trees, and support vector machines have been successfully applied to improve the accuracy and reliability of weather forecasting. These algorithms are capable of learning from historical weather patterns and adapting to new data inputs, which allows them to provide predictive insights with greater precision than traditional methods.

7.3 IOT MODEL DESCRIPTION

In an IOT enabled weather monitoring system project, Arduino Uno measures 4 weather parameters using 4 sensors. These sensors are a temperature sensor, humidity sensor, light sensor, and rain level sensor. These 4 sensors are directly connected to Arduino Uno since it has an inbuilt Analog to digital converter. The weather monitoring system gives high accuracy and reliability for weather monitoring and climate changing. It uses renewable energy sources like solar panels for charging the connected battery. Through the web, it accesses real time weather information and data. This system can be communicated over a general packet radio service (GPRS) network. Low maintenance is required for end users. It is capable for storing data and providing it to the users as required.

7.3.1 PREPOSED HARDWARE for WEATHER MONITORING

The implemented system consists of a microcontroller (ESP8266) as a main processing unit for the entire system and all the sensors and devices can be connected with the microcontroller. The sensors can be operated by the microcontroller to retrieve the data from them and it processes the analysis with the sensor data and updates it to the internet through a Wi-Fi module connected with via ThingSpeak then we can measure temperature, humidity, pressure and rainfall.

7.3.1.1 Circuit Diagram

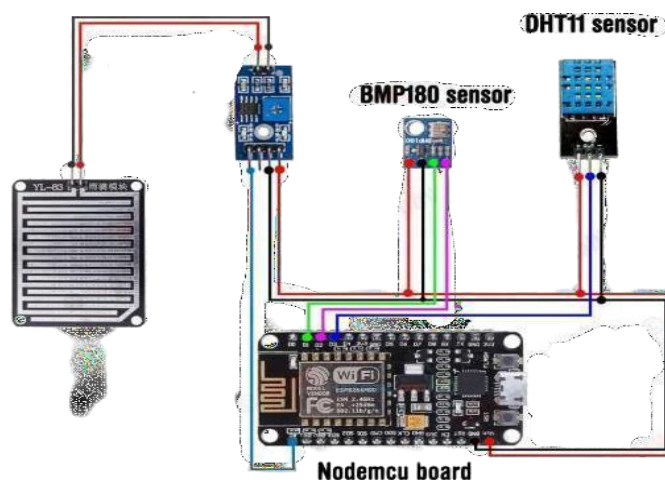


Fig. 1: Circuit Diagram of Hardware Implementation

7.3.1.2 ESP8266 - Wi-Fi Module

The ESP8266 Wi-Fi Module is a self-contained SOC within an integrated TCP/IP protocol stack that can give any microcontroller access to your Wi-Fi network. The ESP8266 is capable of either hosting an application or offloading all Wi-Fi networking functions from another application processor. Each ESP8266 module comes pre-programmed with an AT command set firmware, meaning, you can simply hook this up to your Arduino device and get about as much Wi-Fi- ability as a Wi-Fi Shield offers (and that's just out of the box)! The ESP8266 module is an extremely cost- effective board with a huge, ever -growing, community.

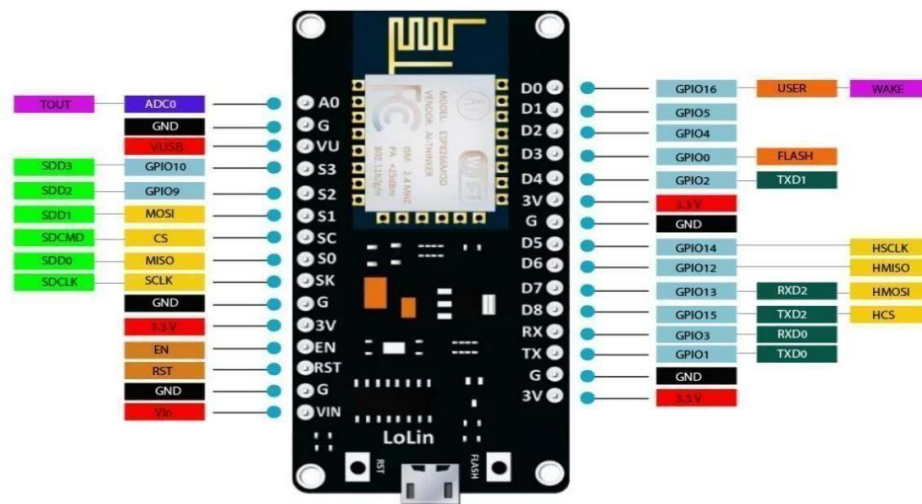


Fig. 2: NodeMCU ESP8266

Features of ESP8266:

- Low cost, compact and powerful Wi-Fi Module
- Power Supply: +3.3V only
- Current Consumption: 100mA
- I/O Voltage: 3.6V (max)
- I/O source current: 12mA (max)
- Built-in low power 32-bit MCU @ 80MHz
- 512kB Flash Memory
- Can be used as Station or Access Point or both combined

7.3.1.3 DHT11 (Temperature & Humidity Sensor)

The DHT-11 Digital Temperature and Humidity Sensor is a basic, ultra-low-cost digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air and spits out a digital signal on the data pin (no analog input pins needed).

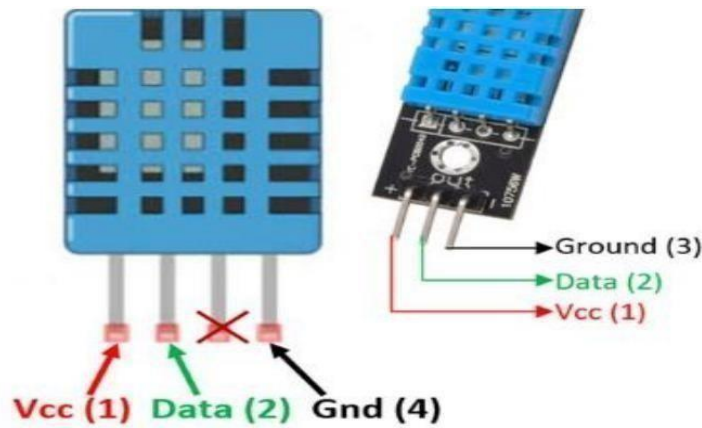


Fig. 3: DHT11 Sensor

DHT11 Specifications:

- Operating Voltage: 3.5V to 5.5V
- Operating current: 0.3mA (measuring) 60uA (standby)
- Output: Serial data
- Temperature Range: 0°C to 50°C
- Humidity Range: 20% to 90%
- Resolution: Temperature and Humidity both are 16-bit
- Accuracy: $\pm 1^\circ\text{C}$ and $\pm 1\%$

7.3.1.4 BMP180 (Pressure Sensor)

BMP180 is one of sensor of BMP XXX series. They are all designed to measure Barometric Pressure or Atmospheric pressure. BMP180 is a high precision sensor designed for consumer applications. Barometric Pressure is nothing but weight of air applied on everything. The air has weight and wherever there is air its pressure is felt. BMP180 sensor senses that pressure and provides that information in digital output.

Also, the temperature affects the pressure and so we need temperature compensated pressure reading. To compensate, the BMP180 also has good temperature sensor.



Fig. 4: BMP180 Pressure Sensor

BMP180 Specifications:

- Operating voltage of BMP180: 1.3V – 3.6V
- Input voltage of BMP180 MODULE: 3.3V to 5.5V
- Peak current: 1000uA
- Consumes 0.1uA standby
- Maximum voltage at SDA, SCL: VCC + 0.3V
- Operating temperature: -40°C to +80°C

7.3.1.5 Rain Sensor

A rain sensor is one kind of switching device which is used to detect the rainfall. It works like a switch and the working principle of this sensor is, whenever there is rain, the switch will be normally closed.

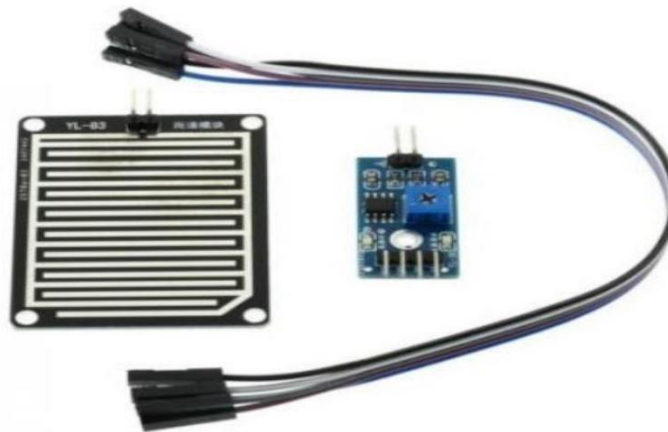


Fig. 5: Rain Sensor

7.3.1.6 Rain Sensor Module

The rain sensor module/board is shown below. Basically, this board includes nickel coated lines and it works on the resistance principle. This sensor module permits to gauge moisture through analog output pins & it gives a digital output while moisture threshold surpasses. This module is similar to the LM393 IC because it includes the electronic module as well as a PCB. Here PCB is used to collect the raindrops. When the rain falls on the board, then it creates a parallel resistance path to calculate through the operational amplifier.



Fig. 6: Rain Sensor Module

Rain Sensor Specifications:

- This sensor module uses good quality of double-sided material.
- Anti-conductivity & oxidation with long time use.
- The area of this sensor includes 5cm x 4cm and can be built with a nickel plate on the side.
- The required voltage is 5V.
- The size of the small PCB is 3.2cm x 1.4cm.
- It uses an LM393 comparator with wide voltage.
- The output of the comparator is a clean waveform and driving capacity is above 15mA.

7.4 RESULT ANALYSIS

The project will involve the development of advanced algorithms and software tools for real- time analysis of weather data. Employing techniques such as statistical analysis, machine learning, and data visualization, these tools will generate actionable insights to support informed decision-making.

Additionally, the system will feature robust reporting capabilities, disseminating weather information to end-users through intuitive interfaces that facilitate easy understanding and interpretation.

CHAPTER-8

CODING

```
#include <SFE_BMP180.h>
#include <Wire.h>
#include <ESP8266WiFi.h>
#include "DHT.h" DHT dht(D3, DHT11);
SFE_BMP180 bmp;
double T, P;
char status;
WiFiClient client;

String apiKey = "2JZ16M23J7RM849B";
const char *ssid = "Mi A2"; //WiFi Name
const char *pass = "pukhraj2004"; //WiFi Password
const char* server = "api.thingspeak.com";

void setup() {
  Serial.begin(115200);
  delay(10);
  bmp.begin();
  Wire.begin();
  dht.begin();
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");

  Serial.println("WiFi connected");
}

void loop() {
```

```

//BMP180 sensor
status = bmp.startTemperature();
if (status != 0) {
  delay(status);
  status = bmp.getPressure(P, T);
  if (status != 0) {
    Serial.println("");
    delay(status);
    Serial.println("");

    status = bmp.getTemperature(T);
    status = bmp.startPressure(3);// 0 to 3
    if (status != 0) {
      delay(status);
      status = bmp.getPressure(P, T);
      if (status != 0) {
        delay(status);
        status = bmp.getPressure(P, T);
        if (status != 0) {
          Serial.println("");
        }
      }
    }
  }
}

```

```

//DHT11 sensor

```

```

float h = dht.readHumidity();
float t = dht.readTemperature();

```

```

if (isnan(h) || isnan(t)) {
  Serial.println("Failed to read from DHT sensor!");
}

```

```
return;
}
```

```
//Rain sensor
```

```
int r = analogRead(A0);
r = map(r, 0, 1024, 0, 100);
```

```
Serial.println("");
```

```
if (client.connect(server, 80))
{
String postStr = apiKey;
postStr += "&field1=";
postStr += String(t);
```

```
postStr += "&field2=";
postStr += String(h);
Serial.println("");
postStr += "&field3=";
postStr += String(P, 2);
```

```
postStr += "&field4=";
postStr += String(r);
postStr += "\r\n\r\n\r\n\r\n";
client.print("POST /update HTTP/1.1\n");
client.print("Host: api.thingspeak.com\n");
client.print("Connection: close\n");
client.print("X-THINGSPEAKAPIKEY: " + apiKey + "\n");
```

```
client.print("Content-Type: application/x-www-form-urlencoded\n");
client.print("Content-Length: ");
client.print(postStr.length());
```

```
client.print("\n\n\n\n");
```

```
client.print(postStr);
```

```
Serial.print("Temperature: ");
```

```
Serial.println(t);
```

```
Serial.print("Humidity: ");
```

```
Serial.println(h);
```

```
Serial.print("absolute pressure: ");
```

```
Serial.print(P, 2);
```

```
Serial.println("mb");
```

```
Serial.print("Rain");
```

```
Serial.println(r);
```

```
}
```

```
client.stop();
```

```
delay(10000);
```

```
}
```

CHAPTER-9

OUTPUT SCREEN AND WORKING

9.1 GETTING REAL-TIME DATA USING SENSORS

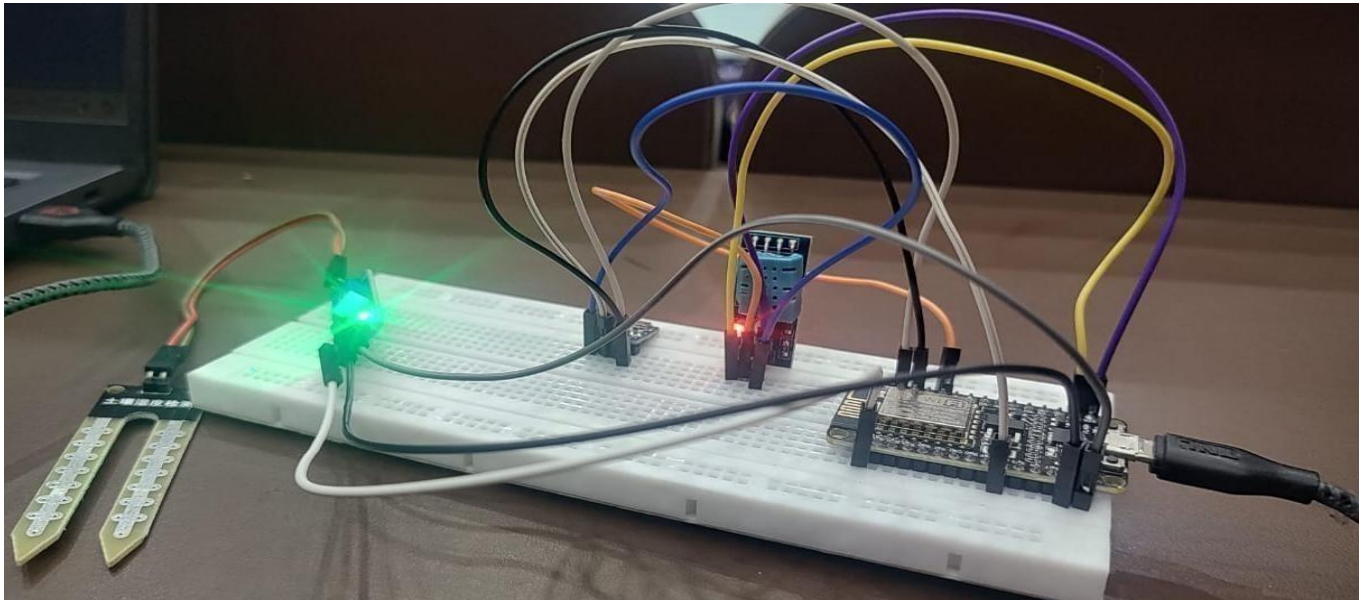


Fig. 9.1 Real-Time Data Using Sensors

Acquiring real-time data using sensors is a streamlined process designed to efficiently capture and transmit environmental information for immediate analysis and decision-making. Initially, sensors equipped with IoT capabilities are strategically deployed across diverse locations, targeting key environmental parameters such as temperature, humidity, air quality, and wind speed.

These sensors continuously collect data and transmit it wirelessly to a central processing unit or data hub. Upon receiving the data, the central processing unit parses and organizes the incoming streams, applying real-time processing algorithms to detect anomalies or patterns. The processed data is then integrated with visualization tools.

Continuous monitoring and maintenance of the system are essential to ensure its reliability and accuracy over time, involving regular calibration of sensors, monitoring of data quality, and troubleshooting of any issues that arise.

9.2 GETTING DATA ON ARDUINO IDE

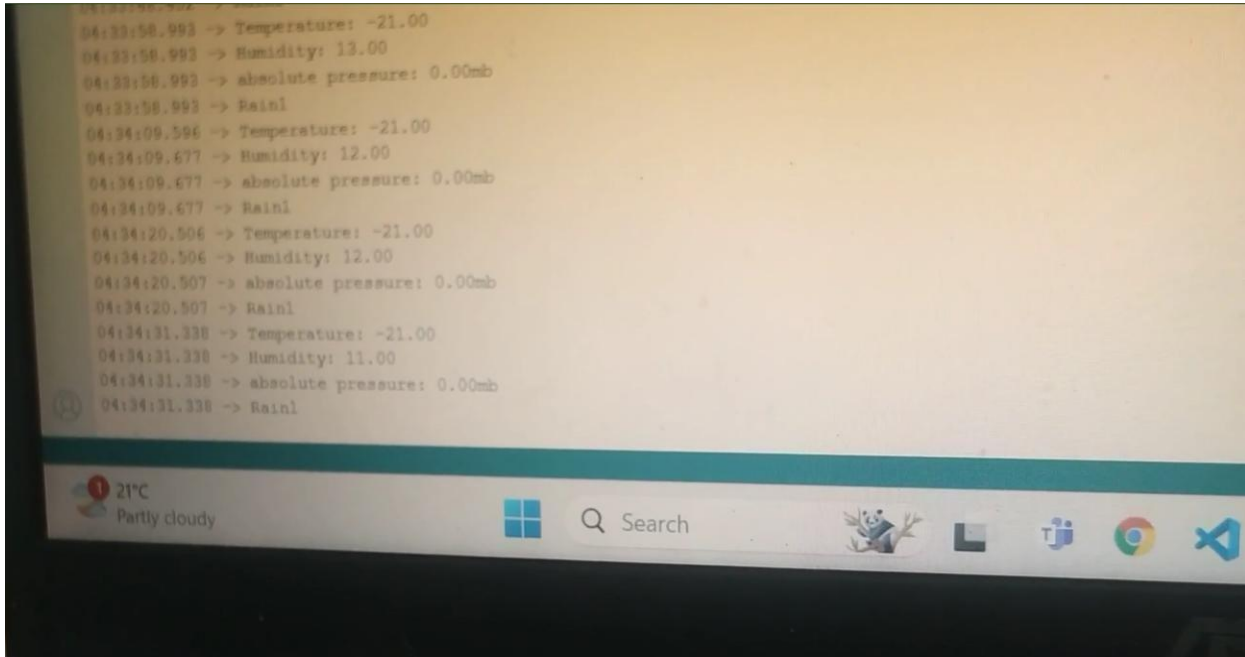


Fig. 9.2 Getting Data on ArduinoIDE from Sensors

To obtain data from sensors using the ESP8266 with Arduino IDE, we follow a streamlined process. First, we configure the ESP8266 to establish a Wi-Fi connection, enabling communication with external devices and online platforms. Next, we develop firmware code within the Arduino IDE to read data from the sensors. This includes initializing the sensors, reading sensor values, and transmitting the data over Wi-Fi. In the Arduino IDE, obtaining data involves configuring the microcontroller to interface with sensors and retrieve information. Initially, sensor modules are connected to the Arduino board, establishing physical connections and ensuring compatibility. Then, using Arduino's programming language and libraries, code is written to initialize sensor communication and capture data readings.

Once the firmware code is developed, we upload it to the ESP8266 microcontroller using the Arduino IDE. The code is compiled and transferred to the ESP8266 over the USB connection, and the microcontroller executes the code upon reset.

Finally, we verify the data transmission process to ensure that sensor data is successfully sent to the intended destination. This may involve monitoring serial output messages from the ESP8266 within the Arduino IDE.

9.3 UPDATING DATA ON THINGSPEAK PLATFORM

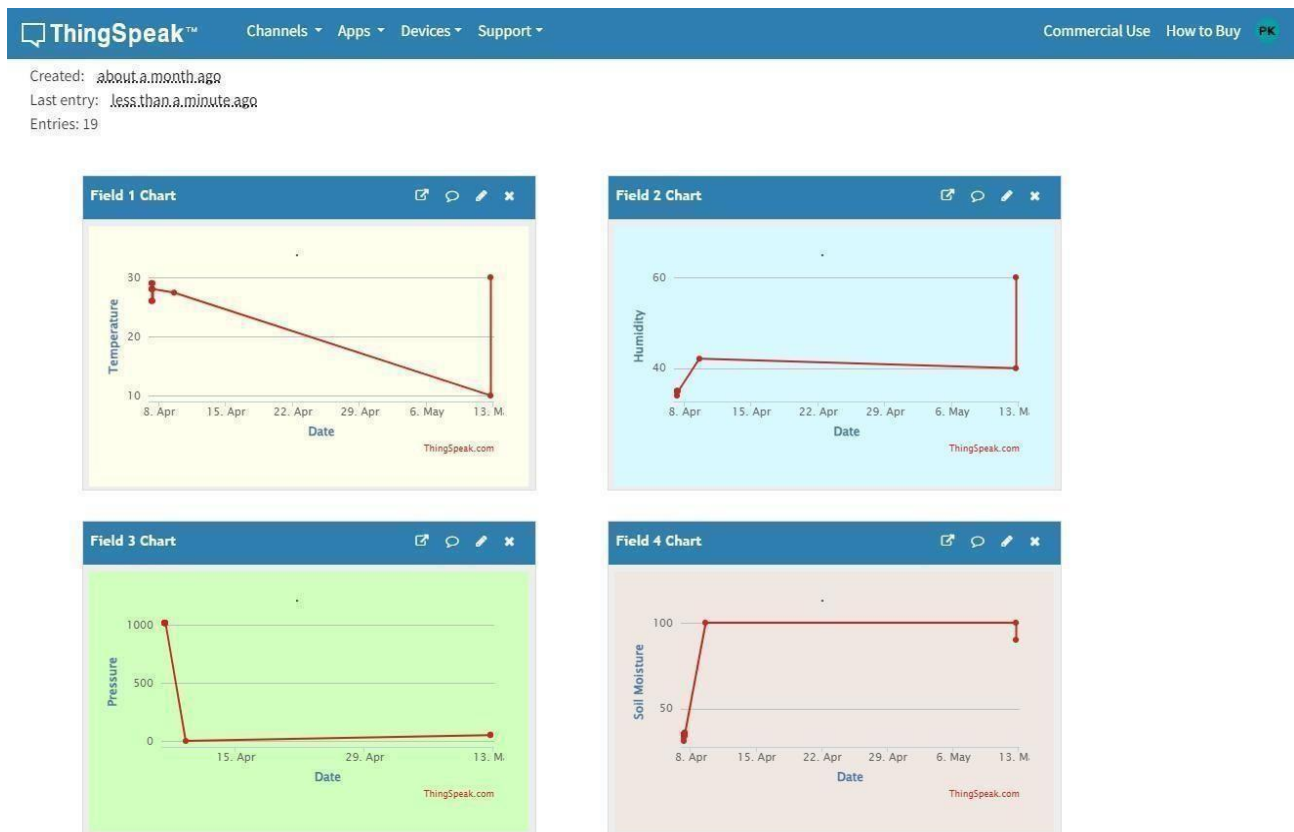


Fig. 9.3 Getting Data on ThingSpeak Platform from ArduinoIDE

Once the data is collected and stored in variables, the Arduino sketch utilizes the ThingSpeak API to send this data to the ThingSpeak platform. The API key provided by ThingSpeak during channel creation is included in the sketch to authenticate the connection between the Arduino and ThingSpeak.

Using the ThingSpeak API, the Arduino sketch constructs an HTTP POST request containing the sensor data to be sent to the ThingSpeak server. This request includes the API key, the specific fields in the ThingSpeak channel where the data should be stored, and the values of the sensor data collected by the Arduino.

Upon receiving the HTTP POST request, the ThingSpeak server processes the incoming data and stores it in the designated fields of the corresponding ThingSpeak channel. The data is time stamped upon arrival, allowing for time-series analysis and visualization on the ThingSpeak platform.

9.4 INTEGRATING DATA ON WEB PLATFORMS

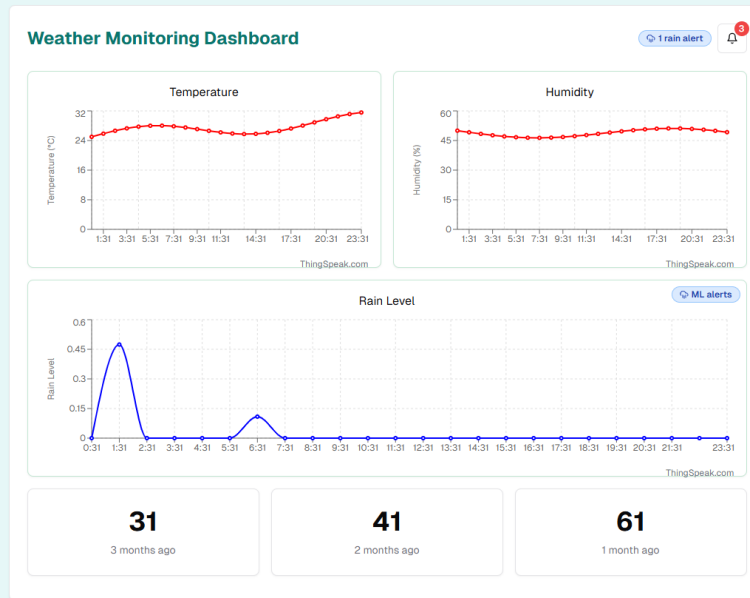


Fig. 9.4 Getting Data on user's Website

Integrating data from ThingSpeak onto our web platform involves leveraging the ThingSpeak API to access real-time weather data securely. Initially, we authenticate our requests using an API key obtained from ThingSpeak, ensuring the confidentiality and integrity of data transmission. Utilizing the ThingSpeak API endpoints, we retrieve the latest weather data from our designated channel, specifying parameters to fetch specific fields such as temperature, humidity, and precipitation. The data is fetched in JSON format, ready for processing and display on our web platform.

Upon retrieval, we parse the JSON response, extracting relevant information to present to users. To enhance user experience, interactive visualizations and widgets are designed, allowing users to explore weather data intuitively. These include charts, graphs, and maps displaying temperature variations, humidity levels, and precipitation patterns across different locations and time periods. Efficiency is optimized through features like data caching and compression, reducing latency and bandwidth usage for faster page load times. Cached data is stored locally on users' devices, while compressed data payloads minimize transmission overhead. Ensuring platform responsiveness and compatibility across devices and browsers is paramount.

9.5 HISTORICAL WEATHER DATA SET OF BHOPAL CITY

01-01-2013 00:00	24	10	0	8.7	4	1	31	9:56 AM
01-01-2013 01:00	24	10	0	8.7	4	1	31	9:56 AM
01-01-2013 02:00	24	10	0	8.7	4	1	31	9:56 AM
01-01-2013 03:00	24	10	0	8.7	4	1	31	9:56 AM
01-01-2013 04:00	24	10	0	8.7	4	1	31	9:56 AM
01-01-2013 05:00	24	10	0	8.7	4	1	31	9:56 AM
01-01-2013 06:00	24	10	0	8.7	4	4	31	9:56 AM
01-01-2013 07:00	24	10	0	8.7	4	5	31	9:56 AM
01-01-2013 08:00	24	10	0	8.7	4	5	31	9:56 AM
01-01-2013 09:00	24	10	0	8.7	4	6	31	9:56 AM
01-01-2013 10:00	24	10	0	8.7	4	6	31	9:56 AM
01-01-2013 11:00	24	10	0	8.7	4	6	31	9:56 AM

Fig. 9.5.1 Historical weather dataset of Bhopal city

The dataset used in this analysis has been sourced from Kaggle, specifically from the "Historical Weather Data for Indian Cities" collection. For this project, we have chosen data for Bhopal City. The dataset was created to address the increasing need for historical weather data, which can be valuable for various research and forecasting purposes. It includes weather data for the top 8 most populous cities in India. The data for these cities was collected using the worldweatheronline.com API and the wwo_hist package.

The weather data spans from 01-01-2013 to 31-01-2024, providing an extensive record of hourly weather data for Bhopal City. The dataset captures a period of over a decade, making it highly useful for understanding long-term weather trends. This data can serve multiple purposes, such as analyzing climate change and its impact on local weather patterns, forecasting future weather conditions, or conducting studies related to global warming.

Key Features of the Dataset:

- **Time Period:** Data collected from 2013 to 2024.
- **City:** Bhopal, Madhya Pradesh.
- **Weather Attributes:** The dataset contains hourly weather data, including attributes such as temperature, humidity, precipitation, wind speed, and more.
- **Purpose:** The dataset is intended for predictive analysis, including weather forecasting for upcoming days, weeks, months, or even seasons.

It is important to note that the data was extracted using the worldweatheronline.com API, and while the dataset is extensive, we cannot guarantee its absolute accuracy.

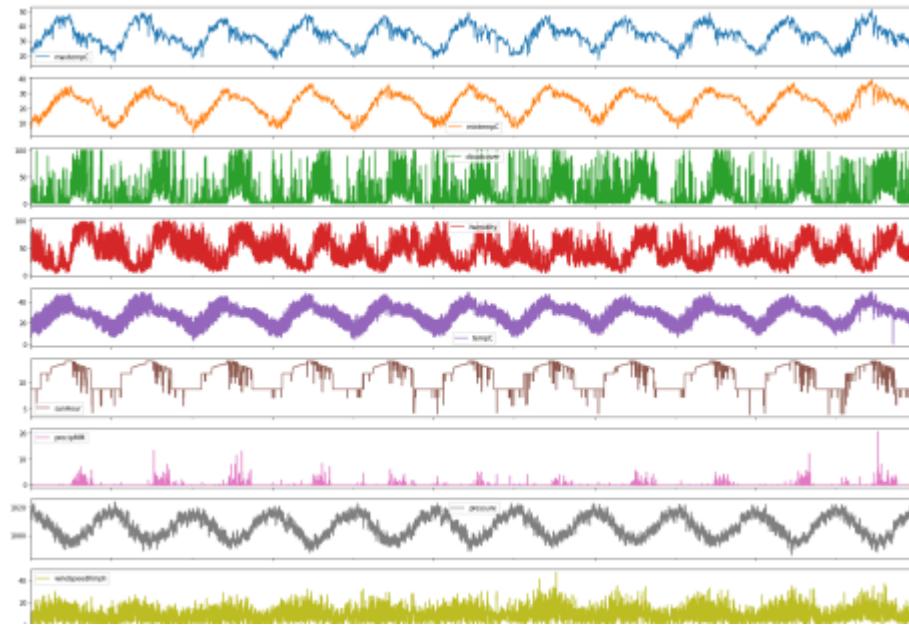


Fig. 9.5.2 Plot for each factor for 11 years

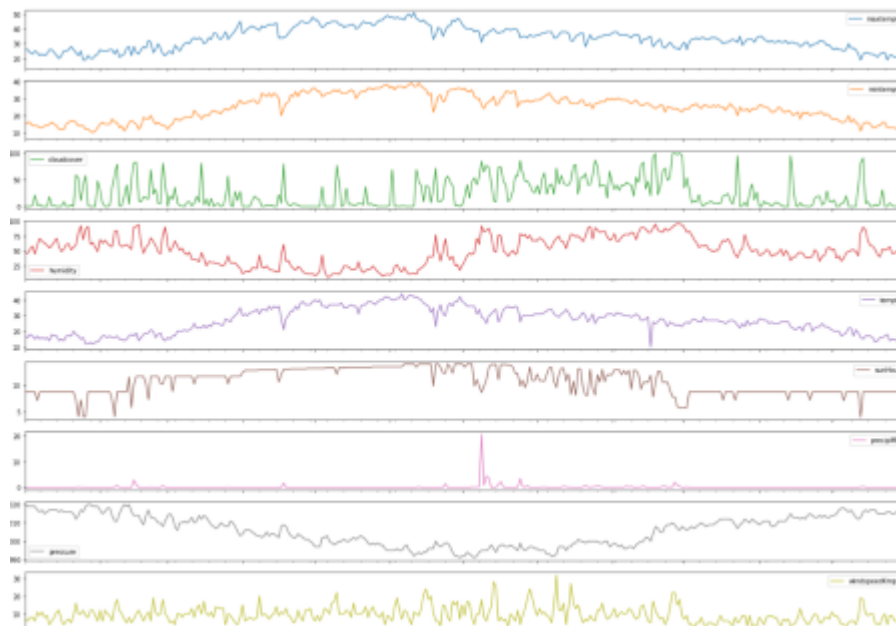


Fig. 9.5.3 Plot for each factor for 1 year

9.6 NOTIFICATION WINDOW

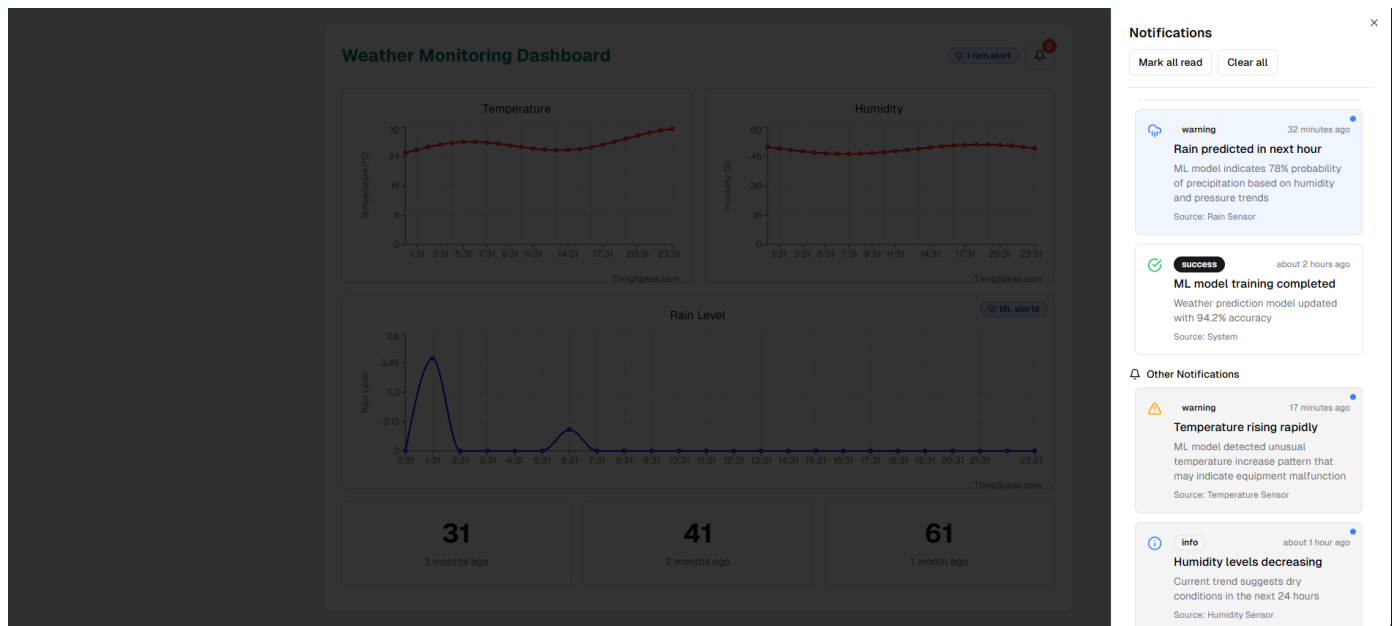


Fig. 9.6 Notification Window

The Notification Window in the weather monitoring system is a crucial feature that ensures users are always informed about significant weather events and emergencies in real time. This window is designed to provide instant alerts about severe weather conditions, such as storms, high winds, extreme temperatures, heavy rainfall, or flood warnings. These notifications are generated by analyzing both live and historical weather data, which is processed through machine learning algorithms to predict potential hazards. When an emergency or critical weather condition is detected, the system triggers an alert that is displayed prominently in the Notification Window.

The design of the notification window prioritizes visibility, ensuring that users can quickly recognize and act upon the alert. Notifications are displayed in a user-friendly format with clear and concise information that includes the nature of the emergency, the expected impact, the geographical area affected, and any recommended actions for safety. This feature not only helps individuals prepare for adverse weather conditions but also assists in making timely decisions, such as evacuating an area or taking shelter.

Furthermore, the Notification Window is adaptable, allowing for different levels of urgency. Critical emergencies will trigger urgent, attention-grabbing alerts, while less severe conditions might prompt a warning or advisory. The system's ability to send real-time notifications based on machine learning predictions and weather patterns ensures that users are kept informed and safe, enhancing the overall effectiveness of the weather monitoring system in managing extreme weather events.

CHAPTER-10

CONCLUSION

10.1 CONCLUSION

The primary goal of this project was to develop an **integrated weather monitoring system** using **IoT technologies** and enhance **weather forecasting capabilities** through advanced **machine learning algorithms**. The first part of the project focused on creating a real-time weather monitoring system powered by IoT-enabled sensors, while the second part aimed to predict temperature trends using historical weather data and various machine learning models.

IoT-Based Weather Monitoring System

The IoT-based weather monitoring system successfully demonstrated how real-time data collection using a network of interconnected sensors could overcome the limitations of traditional weather monitoring methods. By deploying sensors capable of measuring environmental parameters such as temperature, humidity, wind speed, and precipitation, the system provided scalable, flexible, and accessible weather data. This approach not only enhances the accuracy of weather observations but also enables timely responses to weather-related events, improving resilience in sectors like agriculture, transportation, and disaster management.

Machine Learning-Based Weather Forecasting

- In the second part of the project, we applied various **machine learning algorithms**—including **Linear Regression, Polynomial Regression, Decision Tree Regression, and Random Forest Regression**—to predict the temperature of Bhopal City using historical weather data from 2013 to 2024.
- **Linear Regression** proved to be a **low-bias, high-variance model**, sensitive to outliers and less stable in predictions. To improve this model, gathering more data could help reduce variance.
- **Polynomial Regression** showed a **high-bias, low-variance behavior**, indicating that the model may oversimplify complex weather patterns. This limitation suggests that increasing the time window for feature selection (e.g., using data from the past 4–5 days instead of just 2 days) could improve accuracy.
- **Decision Tree Regression** demonstrated good performance but was limited by overfitting in some cases due to its reliance on specific data splits.

- **Random Forest Regression** emerged as the most **accurate and robust model**, benefiting from its ensemble approach, which reduces overfitting and handles complex datasets effectively.

Despite the strong performance of these models, it's important to note that they still faced challenges in predicting long-term weather patterns accurately. However, as data collection methods improve and more sophisticated algorithms are developed, these models have the potential to outperform traditional forecasting tools in the future.

Key Findings and Insights

- **Accuracy and Performance:** Among all models, **Random Forest Regression** consistently outperformed others, demonstrating superior accuracy and robustness in temperature prediction. This aligns with its reputation as a versatile and reliable algorithm in predictive modeling tasks.
- **Model Limitations:** Linear regression, while useful for simple trends, struggled with complex weather patterns due to its sensitivity to outliers. Polynomial regression, though better at capturing non-linear relationships, showed high bias, limiting its predictive power.
- **Practical Applications:** The machine learning models, particularly Random Forest, proved to be valuable for real-time and short-term weather forecasting, with potential applications in agriculture, disaster management, and urban planning.

10.2 FINAL REMARKS

This project highlights the transformative potential of combining **IoT technologies** with **machine learning** to enhance weather monitoring and forecasting. By bridging real-time data collection with predictive analytics, we can create more responsive, accurate, and adaptive weather monitoring systems that address global challenges related to climate change, disaster management, and environmental sustainability.

The synergy between IoT and machine learning offers a promising path forward, paving the way for smarter, more resilient weather monitoring infrastructures that can support a wide range of applications—from urban planning and agriculture to emergency response and climate research.

REFERENCES

WEBSITES (with exact URL up to page)

- [1] <https://aws.amazon.com/whatis/iot/#:~:text=The%20term%20IoT%2C%20or%20Internet,as%20between%20the%20devices%20themselves.>
- [2] <https://www.techtarget.com/iotagenda/definition/Internet-of-Things-IoT>
- [3] <https://www.geeksforgeeks.org/introduction-to-internet-of-things-iot-set-1/>
- [4] Technologies, IEEE, (2016). [3]. Ravi Kishore Kodali and Archana Sahu “An IoT based Weather Information Prototype Using We Mos, IEEE, (2016)
- [5] <https://www.hackster.io/pranavk2020/iot-based-weather-station-using-thingspeak-iot-analytics-ef1174>
- [6] Kavya Ladi, A V S N Manoj, G V N Deepak, “IOT Based Weather Reporting System to Find Dynamic Climatic Parameters”, (2017)
- [7] <https://srituhobby.com/iot-based-weather-monitoring-system-using-nodemcu-and-thingspeak/>
- [8] www.ir.juit.ac.in:8080/jspui/bitstream/123456789/10241/1/Weather%20Reporting%20System%20using%20IoT.pdf
- [9] https://www.rcciit.org/students_projects/projects/ee/2022/GR9.pdf

PROJECT SUMMARY

About Project

Title of the project	WEATHER FORECASTING SYSTEM
Semester	8th
Members	3
Team Leader	Aleena Khanam
Describe role of every member in the project	Aleena Khanam (Hardware requirements, Documentation) Diksha Rupani (Frontend, ML model training) Baishnavi Sharma (System Architect, Hardware integration, Notification System UI development)
What is the motivation for selecting this project?	This project aims to overcome the limitations of traditional weather monitoring by developing an IoT-based system that delivers real-time, localized environmental data. By using smart sensors to track parameters like temperature, humidity, and wind speed, the system provides accurate, continuous monitoring. Inspired by modern alert dashboards, it also includes a built-in notification system that sends timely alerts via email, SMS, or push notifications when weather anomalies are detected. This ensures users receive not only raw data but also actionable insights to support fast, informed decisions.
Project Type (Desktop Application, Web Application, Mobile App, Web)	IOT Web Application

Tools & Technologies

Programming language used	C++ and Python
IDE used (with version)	Arduino IDE 2.3.3 VS Code 1.98
Front End Technologies (with version, wherever applicable)	Html (5) and CSS (3)
Back End Technologies (with version, wherever applicable)	API
Database (Yes / No)	No

Software Design & Coding

Total no. of frontend pages	<i>1 page</i>
Front end validations applied (Yes / No)	No
Session management done (in case of web applications)	No
Is application browser compatible (in case of web applications)	Yes
Commenting done in code (Yes / No)	Yes
Exception handling done (Yes / No)	No
Naming convention followed (Yes / No)	Yes
What difficulties faced during deployment of the project?	NA
Total no. of Use-cases	NA

Project Requirements

MVC architecture followed (Yes / No)	YES
If yes, write the name of MVC architecture followed (MVC-1, MVC- 2)	MVC-2
Design Pattern used (Yes / No)	YES
If yes, write the name of Design Pattern used	Observer Pattern
Interface type (CLI / GUI)	GUI
Total no. of Functional Requirements	4
List few important non-Functional Requirements	Portability, Flexibility, Maintainability, Compatibility
Number of Actors	NA

Testing

Which testing is performed? (Manual or Automation)	Manual
Is Beta testing done for this project?	YES

Write project narrative covering above mentioned points.

The **IoT-based Weather Forecasting System** is an innovative solution that utilizes a network of Internet of Things (IoT) sensors to collect real-time environmental data such as temperature, humidity, wind speed, and atmospheric pressure. These sensors are deployed in various geographical locations to provide highly localized, accurate weather information. The collected data is transmitted to a cloud platform for processing and analysis, where machine learning algorithms and statistical models generate weather forecasts. Users can access these forecasts and real-time weather conditions through a web or mobile application, which also provides alerts for extreme weather events such as storms or heatwaves. This system is designed to improve decision-making for a range of users, including farmers, emergency responders, and urban planners, by offering timely, reliable, and hyper-localized weather data that can inform activities like crop management, disaster preparedness, and resource allocation.

Name	Enrollment	Guide Signature
Aleena Khanam	0187CS211018	(Prof. Shumali Gupta)
Baishnavi Sharma	0187CS211048	
Diksha Rupani	0187CS211065	

APPENDIX-1

GLOSSARY OF TERMS

(In alphabetical order)

C

CSS: CSS is an acronym that stands for Cascading Style Sheets which is a style sheet language that is used to express how a structured document like an HTML document should look. Together with HTML and JavaScript, CSS is one of the cornerstones of the World Wide Web.

G

GitHub: GitHub is a Git repository hosting service that provides a web-based graphical interface. It is the world's largest coding community. Putting a code or a project into GitHub brings it increased, widespread exposure. Programmers can find source codes in many different languages and use the command-line interface, Git, to make and keep track of any changes.

H

HTML: HTML is an acronym for Hypertext Markup Language and is the standard language for documents that have been designed to be displayed in a web browser. You have certainly met HTML in one way or another, as all web pages on the Internet are written using a version of HTML.

I

IDE: IDE is an acronym for Integrated Development Environment. It is the software suite used by developers in a development environment and is designed to maximize productivity and efficiency for the developer.

J

JS: JavaScript is a lightweight, interpreted programming language. It is designed for creating network-centric applications. It is complementary to and integrated with Java. JavaScript is very easy to implement because it is integrated with HTML. It is open and cross-platform.

V

VS Code: Visual Studio Code is a streamlined code editor with support for development operations like debugging, task running, and version control.

T

ThinkSpeak: ThingSpeak, an Internet of Things (IoT) platform renowned for its capabilities in data collection and visualization. Utilizing ThingSpeak's intuitive interface, users create dedicated channels to receive and store data from IoT devices.